



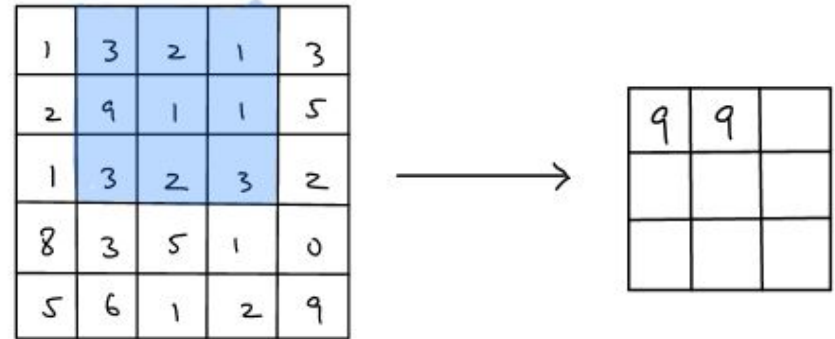
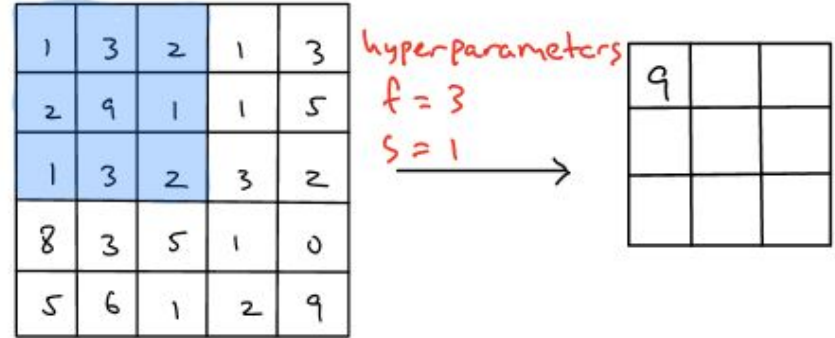
Pooling, Normalisation and Activation Functions

Michael Vella (michael.vella.20@um.edu.mt)

Topic 1: Pooling

The Concept

- A deterministic downsampling operation.
- Contains no learnable parameters (no weights or biases to update via gradient descent).
- Sole purpose: summarise features and reduce dimensions.



Pooling example. Image reproduced from
<https://guandi1995.github.io/Pooling-Layers/>

Pooling - Explained



Hyperparameters

- **Filter size (f):** Sliding window dimensions.
 - Typically 2x2 or 3x3.
- **Stride (s):** The shift distance of the window.
 - Typically 2.
- **Padding (p):** Border pixels.
 - Typically 0. Rarely used in pooling.

The Mathematics

For a feature map with the following input dimensions: $n_h \times n_w \times n_c$

Output Height: $((n_h - f) / s) + 1$

Output Width: $((n_w - f) / s) + 1$

Note: The number of channels (n_c) remains the same.

Trade-off warning: Pooling accelerates training and controls overfitting but excessive pooling can cause critical information to be lost and over-smoothing of the image.

Pooling - Main Types

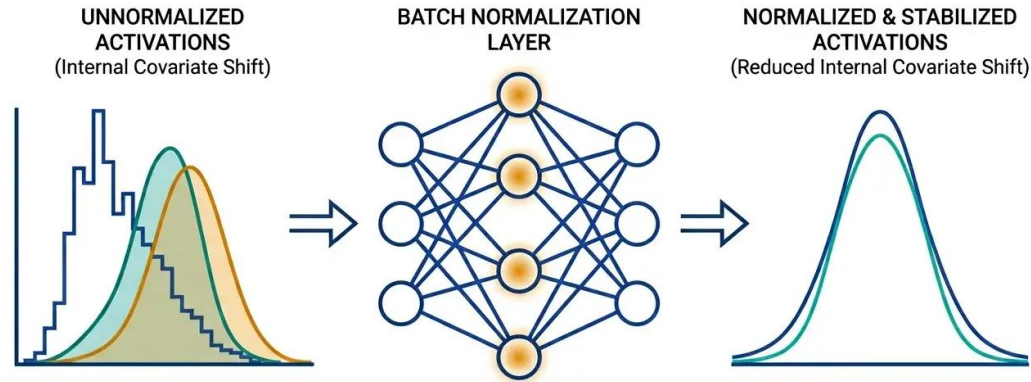
Let's apply different types of pooling to the following feature map input example.

8	1	3	4
3	4	6	7
9	2	1	4
5	4	2	5

Operation	Visual Metaphor	Best Use Case				
Max Pooling (f=2, s=2)	<table><tr><td>8</td><td>7</td></tr><tr><td>9</td><td>5</td></tr></table>	8	7	9	5	Preserving prominent features, edge detection.
8	7					
9	5					
Average Pooling (f=2, s=2)	<table><tr><td>4</td><td>5</td></tr><tr><td>5</td><td>3</td></tr></table>	4	5	5	3	Preserving overall context.
4	5					
5	3					
Global Max Pooling	<table><tr><td>9</td></tr></table>	9	Feature presence detection across the whole image.			
9						
Global Average Pooling	<table><tr><td>4.25</td></tr></table>	4.25	Summarise features at the end of the network.			
4.25						

Topic 2: Batch Normalisation

BATCH NORMALIZATION: STABILIZING NEURAL NETWORKS



Batch Normalisation. Image reproduced from <https://sesen.ai/blog/batch-normalisation-from-scratch>

Batch Normalisation - Explained

Hyperparameters:

- **Batch size (m):** Size of the mini-batch used during training.

Learned parameters:

- **Beta (β):** Parameter to offset normalisation.
- **Gamma (γ):** Used to scale normalisation
- **Moving mean (μ).**
- **Moving variance (σ^2).**

Both the moving mean and variance are calculated during training but only used during inference. Beta and gamma are learnt through backpropagation.

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\};$

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

Batch Normalisation - Uses and limitations



Uses:

- **Reduces internal covariate shift:**
 - Statistical distribution of inputs to a layer changes as the weights of the preceding layers update during training.
- **Higher learning rates:**
 - Data is normalised so we can use larger learning rates.
- **Faster Convergence:**
 - Activations are kept in a stable range which prevents gradients from exploding or vanishing. This drastically reduces the number of training iterations required.

Limitations:

- **Discrepancy between training and inference:**
 - Training -> normalisation performed on each mini-batch.
 - Inference -> using moving statistics.
 - Prone to data distribution changes.
- **Batch size:**
 - Works better with larger batch sizes.
 - Noisy statistics with smaller batch sizes.
- **Computational overhead:**
 - Requires additional computation to calculate mini-batch statistics.

Topic 3: Activation Functions

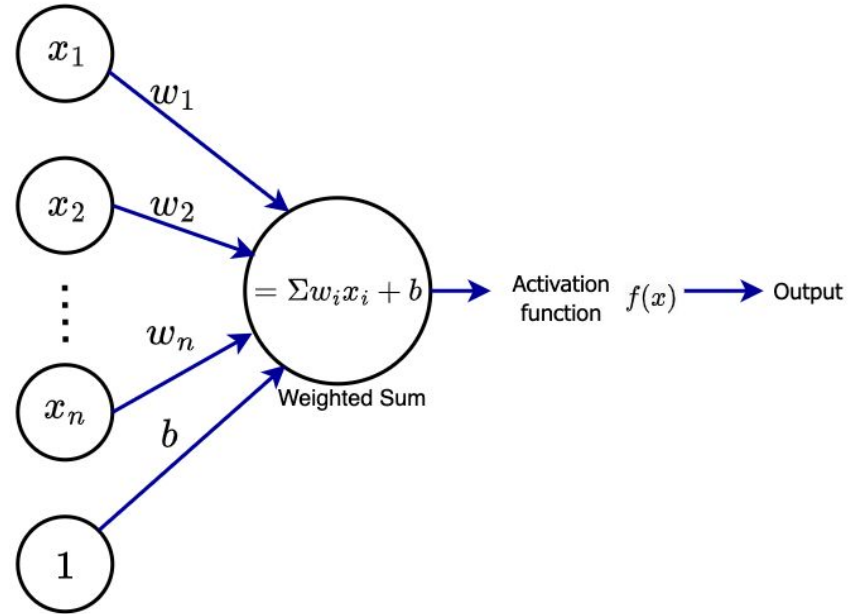


Image reproduced from <https://intuitivetutorial.com/2023/07/23/activation-functions-in-deep-learning/>

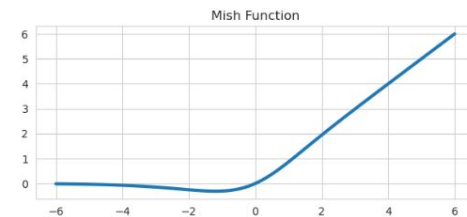
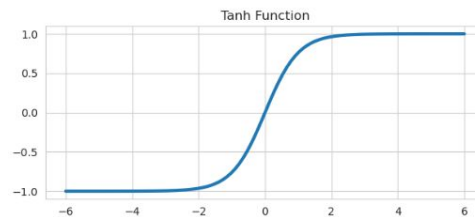
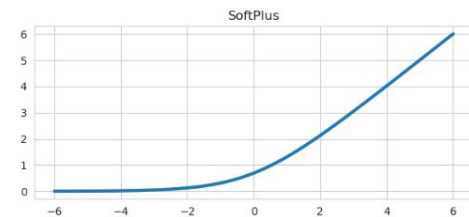
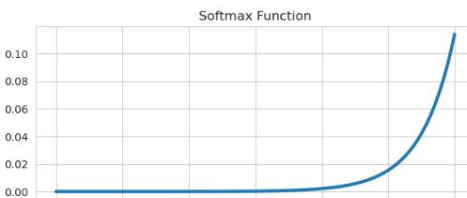
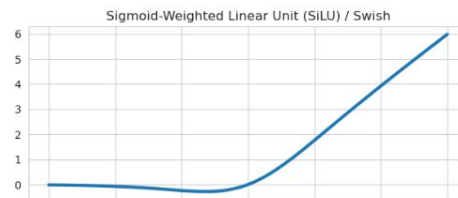
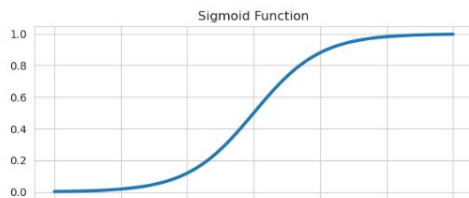
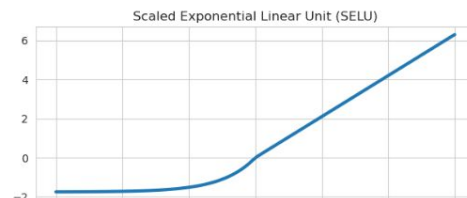
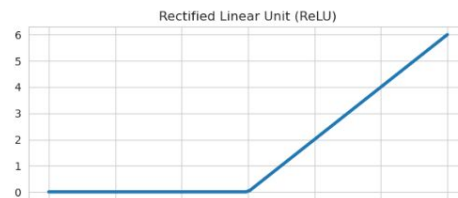
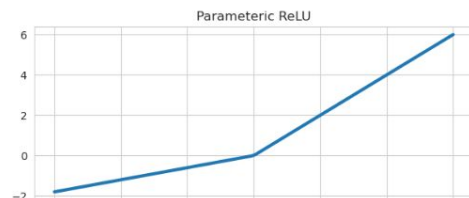
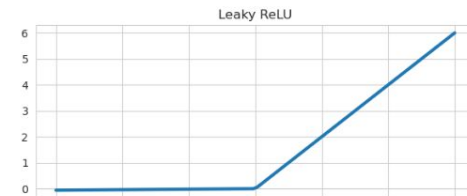
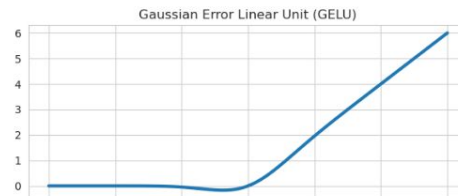
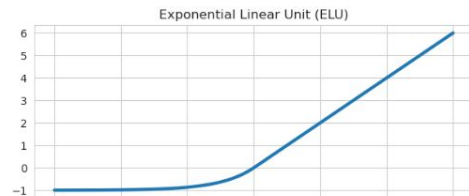
Activation Functions - Essential Properties



A robust activation function typically possesses the following traits:

- **Non-linearity:**
 - The most important property as without it the network is just a series of linear models regardless of how many layers it has.
 - This property is what makes activation functions learn complex patterns in the data.
- **Differentiability:**
 - Must be differentiable so that gradients can flow backward through the network via backpropagation.
- **Computational Efficiency:**
 - Millions of calculations are performed so the function (and its derivative) should be inexpensive.
- **Zero-Centered (Optional but desirable):**
 - Zero centered functions help make gradient updates more stable and efficient.

Activation Functions



Activation Functions - Limitations

The Dying ReLU

If a weight is updated in a way that the input to the neuron remains negative for all inputs, the neuron outputs zero and its gradient becomes zero, permanently deactivating or "killing" the neuron.

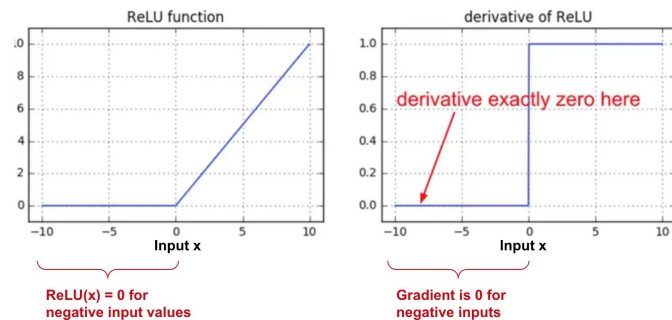


Image reproduced from:

<https://aiml.com/what-is-dying-relu-or-dead-relu-and-why-is-this-a-problem-in-neural-network-training/>

Vanishing Gradient

For very large or very small input values, the curve flattens out, pushing the derivative toward zero. During backpropagation, this causes gradients to shrink, severely slowing or halting learning in deeper layers.

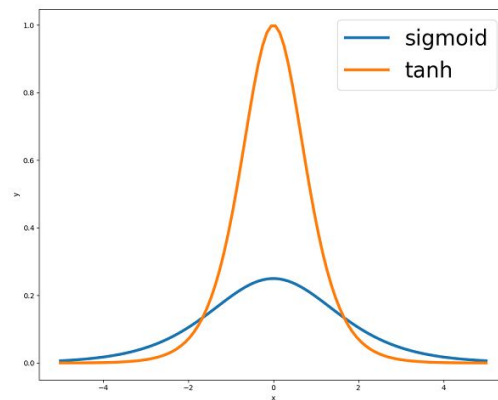


Image reproduced from
<https://stats.stackexchange.com/questions/101560/tanh-activation-function-vs-sigmoid-activation-function>

THANK YOU

Email: michael.vella.20@um.edu.mt

GitHub: <https://github.com/michael-vella/ARI5118-deep-learning-for-computer-vision>

Simulator: <https://mvella-uom-ari5118-deep-learning.streamlit.app/>

Quiz: <https://forms.gle/wfi29gANVJvkrnVb8>

